DAY 48

Deploying Machine Learning Models with Streamlit

Practical guide | Python, Streamlit, Scikit-learn, NumPy and Joblib

Learning outcome: Build an interactive web interface that loads a trained ML model, accepts user input, validates it, generates a prediction, and displays the result.

Project: Diabetes dataset regression demonstration. The dataset and model are for learning only. The output is not a diagnosis or medical advice.

1. What is Streamlit?

Streamlit is a Python framework for building interactive data applications. It is useful for ML demos, analytics dashboards, data exploration tools, and internal applications. It provides interface components directly in Python, so you can build a working UI without writing a large amount of HTML, CSS, or JavaScript.

Roman Urdu: Streamlit Python ki madad se interactive web app banane ka framework hai. Aap input fields, buttons, charts aur prediction results asaani se display kar sakte hain.

2. How the workflow works

• User enters feature values in the Streamlit interface.

• The application validates the values and checks the expected feature count.

• NumPy converts the values into a two-dimensional array.

• The saved Scikit-learn model generates a prediction.

• Streamlit displays the prediction to the user.

Important: A deployed application should reuse a trained model. It should not retrain the model every time a user clicks Predict.

3. Flask vs Streamlit

Area	Flask	Streamlit
Primary use	Web backends and APIs	Interactive data apps
UI components	Usually build or integrate	Built-in Python components
API routes	Strong route control	Not its primary purpose
ML dashboard	Possible	Convenient

4. Install the libraries

```
python -m pip install streamlit scikit-learn joblib numpy
streamlit --version
```

If the streamlit command is not recognized, try python -m streamlit --version. Install packages in the same Python environment used to run the app.

5. Important Streamlit components

st.title(): Main page title.

st.header() / st.subheader(): Section headings.

st.write(): Text and object output.

st.number_input(): Numerical input.

st.text_input(): Text input.

st.button(): Clickable button.

st.success() / st.error(): Status and error messages.

st.metric(): Key numerical result.

st.columns(): Column layout.

st.expander(): Expandable details.

st.cache_resource: Cache a resource such as a loaded model.

6. The dataset and model

The practical uses Scikit-learn's built-in diabetes regression dataset. It has ten numerical input features and a numerical target representing a measure of disease progression. A RandomForestRegressor learns a relationship between the input features and target.

The data is split into training and testing sets. The model learns from the training set. Mean Absolute Error (MAE) measures the average absolute difference between actual and predicted test values. Lower MAE generally indicates smaller errors on that test set, but it does not guarantee performance on new populations.

```
mae = mean_absolute_error(y_test, predictions)
print("Test MAE:", mae)
```

7. Project structure

```
Day-48-Streamlit-ML-Prediction-App.py
diabetes_model.joblib   # created automatically on first run
```

The supplied Python file is self-contained. On first launch, it trains a Random Forest model and saves it beside the script. On later launches, it loads the saved model.

8. Run the project

```
python -m pip install streamlit scikit-learn joblib numpy
streamlit run Day-48-Streamlit-ML-Prediction-App.py
```

Streamlit prints a local URL, commonly http://localhost:8501. Open it in your browser. Enter ten values and select Generate prediction.

9. Code explained

Model loading: joblib.load() reads a saved Python object. The project uses st.cache_resource so the model resource can be reused across Streamlit reruns.

```
saved_model = joblib.load(MODEL_PATH)
model = saved_model["model"]
```

Input conversion: NumPy converts the ten values into a numeric array. Scikit-learn expects a two-dimensional input where rows are samples and columns are features.

```
input_array = np.asarray(values, dtype=float)
prediction = model.predict(input_array.reshape(1, -1))
```

Result display: st.metric() shows the prediction in a clear format. Two decimal places improve readability, but do not mean the prediction is accurate to two decimal places.

```
st.metric("Predicted target value", f"{prediction[0]:.2f}")
```

10. Common errors and solutions

ModuleNotFoundError: Install the missing package in the Python environment used by Streamlit.

Model file missing: The supplied project creates the model on its first run. Check folder permissions if saving fails.

Wrong feature count: This model expects exactly ten values.

Wrong feature order: Keep the same feature order used during training.

Unexpected values: The dataset features are scaled numerical values, not ordinary real-world units.

Port already in use: Stop the other Streamlit process or select another port.

11. Practice tasks

• Change the application title and add your own description.

• Add a sidebar using st.sidebar.header() and st.sidebar.write().

• Display the stored test MAE with st.metric().

• Add an expandable section explaining the model workflow.

• Experiment with different feature values and observe how the output changes.

12. Example solutions

Sidebar:

```
st.sidebar.header("About this project")
st.sidebar.write("A beginner-friendly regression demo.")
```

Display a metric:

```
st.metric("Test MAE", f"{mae:.2f}")
```

Validate shape: A single sample with ten features must have shape (1, 10). The project reshapes a one-dimensional list into this format before prediction.

13. Knowledge check with answers

• What is Streamlit used for? Answer: Building interactive data applications.

• Which command launches the app? Answer: streamlit run app.py.

• Which component collects numerical input? Answer: st.number_input().

• What does joblib.load() do? Answer: Loads a saved Python object, such as a trained model.

• Why must feature order remain consistent? Answer: The model learned a meaning for each feature position during training.

14. Key takeaways

• Streamlit creates interactive data applications using Python.

• A saved model can be reused to generate predictions.

• Input count, order, and representation must match the training setup.

• NumPy prepares model input in the expected shape.

• A responsible ML interface explains the limits of its predictions.

15. References

Streamlit documentation: https://docs.streamlit.io/

Streamlit API reference: https://docs.streamlit.io/develop/api-reference

Scikit-learn: https://scikit-learn.org/stable/

Diabetes dataset: https://scikit-learn.org/stable/datasets/toy_dataset.html#diabetes-dataset

Joblib: https://joblib.readthedocs.io/

Completion target: Run the application and explain how data moves from the Streamlit input fields to the model and back to the displayed prediction.